Progress Report

October 6, 2026

1 Progress since the last meeting

In the last meeting, we discussed starting with a small pilot study before running a large number of simulations. The main idea was to first check whether the fault modelling, data generation and machine learning pipeline work together on a smaller dataset.

The workflow discussed was roughly:

Fault modelling → Generate data → Test one ML method → Analyse the results → Scale up

Based on this, I focused mainly on getting this complete pipeline working rather than generating a large dataset immediately.

1.1 Fault modelling

I first worked on five FSED cases:

- stuck actuator,
- poor balancing,
- excessive control margin,
- clogged filter,
- heat exchanger fouling.

Before combining them into one dataset, I tested the five cases separately. The purpose was mainly to check that introducing a fault actually changes the expected hydraulic or thermal behaviour. For example, for the excessive control margin case, the plant head increased from approximately 29.66 m to 35.60 m when using a head multiplier of 1.20. For the clogged filter case, the mean filter head loss increased from approximately 0.0205 m to 0.409 m. For the heat exchanger fouling case, a UA multiplier of 0.85 was used and changes were observed in the thermal response. For the stuck actuator case, the applied flow became fixed after the fault onset. All five individual validation tests passed before I continued with the combined pilot dataset.

1.2 Pilot dataset

After validating the five cases, I generated a small pilot dataset.

For this first test I used:

- 1 simulation seed,
- 31 consumers,

- consumer 25 as the target consumer,
- 5 FSED cases,
- 1 severity level for each FSED,
- 5 healthy/fault counterfactual pairs,
- 10 physical scenarios in total,
- 48 timesteps for each scenario.

This resulted in 480 observations. All 10 simulations converged. The severity levels used for this pilot were:

FSED	Parameter	Pilot value
Stuck actuator	onset fraction	0.50
Poor balancing	flow multiplier	0.80
Excess control margin	head multiplier	1.20
Clogged filter	filter K	20.0
HX fouling	UA multiplier	0.85

These values are currently used as controlled simulation values for the pilot and not as general fault thresholds.

1.3 Checking the dataset before ML

Before training the ML model, I checked the generated columns because some of them contain information that would directly reveal which fault was simulated. For example, I did not use the FSED name, severity value, scenario ID, pair ID or fault state as ML inputs. I also removed the filter-specific variables from the ML inputs because they only exist for the clogged-filter case. In that situation, even the missing values could indirectly tell the model which fault it is seeing. For the first ML test I therefore used the following measurements:

- desired flow,
- realised flow,
- supply pressure,
- return pressure,
- ETS differential pressure,
- supply temperature,
- return temperature,
- temperature difference,
- delivered load,
- plant head.

1.4 First ML test

For the pilot I selected an MLP as the first ML method. The idea here was not yet to find the best possible model. I wanted to check whether the generated data could go through the complete ML pipeline and whether the selected measurements contain enough information to separate the simulated conditions. During the first test I found a problem with the stuck-actuator data.

The fault starts at 14,400 seconds, but initially I was treating the whole stuck-actuator scenario as faulty. This meant that the observations before 14,400 seconds were labelled as stuck actuator even though the actuator had not become stuck yet.

After correcting this, the fault-active dataset contained 216 observations:

- 48 poor balancing,
- 48 excessive control margin,
- 48 clogged filter,
- 48 HX fouling,
- 24 active stuck-actuator observations.

I then used 5-fold cross-validation as an exploratory test. The five accuracy results were:

$$0.682, \quad 0.651, \quad 0.698, \quad 0.721, \quad 0.744$$

which gave:

$$\text{Mean accuracy} = 0.699$$

and

$$\text{Mean macro-F1} = 0.720.$$

This is currently only a pilot result. Since there is only one independent seed, observations from the same simulation can appear in different folds. Therefore I am not treating this as a final generalisation result.

1.5 Normal vs fault detection

I also tested whether the same MLP could first distinguish normal behaviour from an active fault.

For this test there were 480 observations in total:

- 264 healthy or not-yet-faulted observations,
- 216 active-fault observations.

Again I used 5-fold cross-validation. This means that in each fold approximately 384 observations were used for training and 96 observations for testing.

The results were:

Metric	Result
Accuracy	0.800 ± 0.037
Fault precision	0.980 ± 0.026
Fault recall	0.570 ± 0.096
Fault F1	0.714 ± 0.073

The confusion matrix was:

	Predicted normal	Predicted fault
Actual normal	261	3
Actual fault	93	123

One interesting result from this test is the difference between precision and recall. When the model predicts a fault, it is usually correct. But, it still misses quite a large number of the active-fault observations. This is something I want to investigate when the experiment is expanded.

2 What I learned from the pilot

The main purpose of this pilot was to find problems before generating a much larger dataset. A few things became clear from this test. First, the timing of the fault matters. The stuck-actuator case showed that a scenario should not automatically be considered faulty at every timestep just because a fault is introduced later in that simulation. Second, I need to be careful about data leakage. Some simulation variables can make the classification artificially easy because they directly contain information about how the scenario was generated. Third, the current results should not be treated as final ML performance. The dataset currently contains only one independent seed and one target consumer. I also noticed that the healthy simulations have a relatively high unmet-load rate (around 0.71 in the pilot). I want to check whether this is expected from the current network setup before producing the final large dataset.

3 Next step

During week 42 I plan to move from this small pilot to the larger experiment. The first task is to expand the dataset using more independent simulation conditions. This will include more seeds and severity levels, and I also want to introduce different fault locations so that localisation can later be tested. Most importantly, I will define the training and testing split at the scenario level. The current pilot uses timestep-level cross-validation. For the larger experiment, complete simulation scenarios should remain outside the training set. This will make it possible to test whether the ML method can recognise faults in genuinely unseen simulation cases.

I will first use the MLP on this expanded dataset so that I have one consistent baseline method and a clear answer for how much data was generated, how much was used for training, and how much was used for testing. After this baseline experiment is stable, the next step will be to compare it with a model that can use the network structure and temporal behaviour. The physics-informed extension will be considered after checking which hydraulic variables are available for defining a physically meaningful constraint.

4 Code organisation

At the moment the simulation and experiments have been moved from the development notebook into separate Python files. During week 42 I will clean this into one reproducible project folder, including the experiment scripts, configuration files, required inputs, results and a README describing how the experiments are run. After cleaning the structure, I will upload this version to the shared OneDrive for review rather than uploading the current intermediate working files. The goal after week 42 is to have the first research-question experiment in a much more complete form and then start the pilot work for the next part of the thesis.